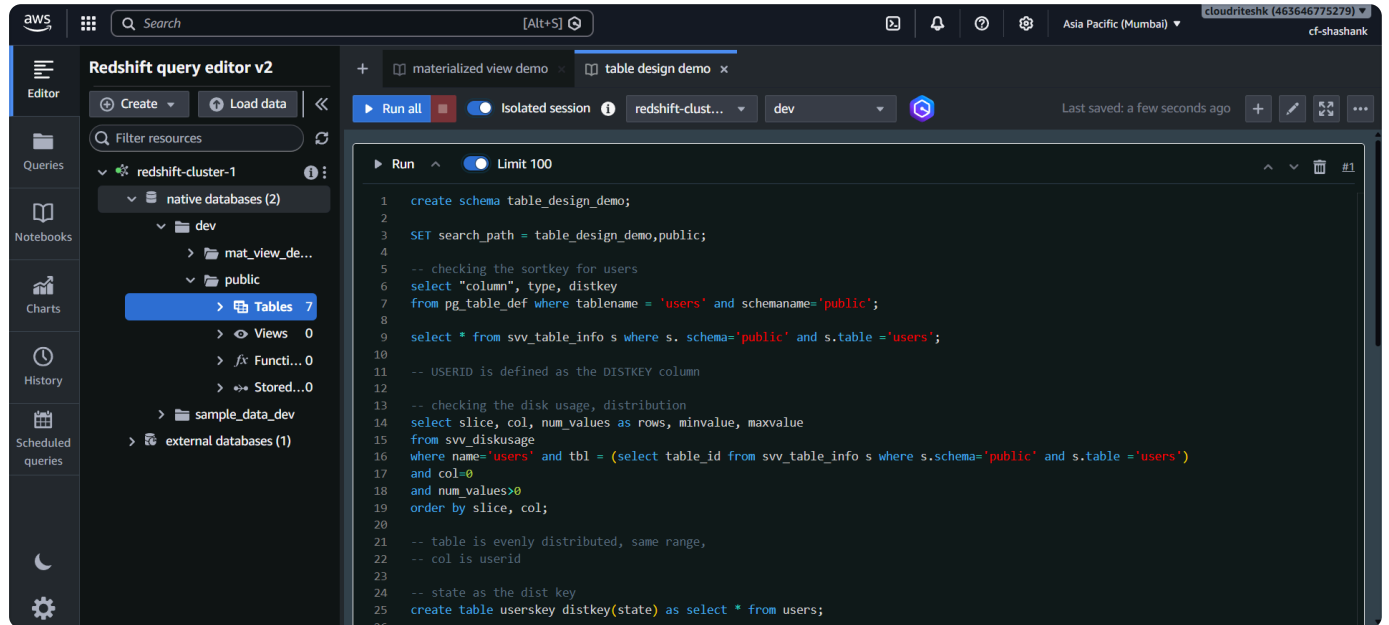


Demo: Table Design - Distribution Style

Summary

Created multiple Redshift tables using KEY, EVEN, ALL, and AUTO distribution styles, analyzed storage distribution using system tables, and observed how different distribution methods affect slice-level data balancing and storage behavior.

Create Demo Schema



Create Schema

```
CREATE SCHEMA table_design_demo;
```

Set Search Path

```
SET search_path TO table_design_demo;
```

Check Existing Table Distribution

Target table:

```
public.users
```

Query Distribution Key using pg_table_def

Editor

Filter resources

redshift-cluster-1

native databases (2)

dev

public

Tables 7

category

date

event

listing

sales

users

venue

Views 0

Functions 0

Stored proc...0

table_design_demo

sample_data_dev

materialized view demo

table design demo x

Run all

Isolated session

redshift-clust...

dev

Last saved: 12 minutes ago

49

select slice, col, num_values as rows, minvalue, maxvalue

50

from svv_diskusage

51

where name='usersauto' and tbl = (select table_id from svv_table_info s where

52

and col=0

53

and num_values>0;

54

55

select * from svv_table_info s where s. schema='table_design_demo' and s. tab

Result 1 (18)

Export

Chart

Row 1, Col 31, Chr 1846

column	type	distkey
userid	integer	true
username	character(8)	false
firstname	character varying(30)	false
lastname	character varying(30)	false
city	character varying(30)	false
state	character(2)	false
email	character varying(100)	false
phone	character(14)	false
zipcode	integer	false

Elapsed time: 58 ms Total rows: 18

Run status

1 out of 1 cell succeeded

Start time: 7:01:15 pm (UTC+05:30)

✓ #1

58 ms

```
SELECT tablename,
       "column",
       distkey
FROM pg_table_def
WHERE schemaname = 'public' AND tablename = 'users';
```

Observation

- userid is the distribution key.

Meaning:

- Data is distributed across slices using userid.

Check Distribution Style

Editor

Filter resources

redshift-cluster-1

native databases (2)

dev

public

Tables 7

Views 0

Functions 0

Stored...0

table_design_...

sample_data_dev

external databases (1)

materialized view demo

table design demo x

Run all

Isolated session

redshift-clust...

dev

Last saved: a few seconds ago

Run

Limit 100

1

create schema table_design_demo;

2

SET search_path = table_design_demo,public;

3

4

-- checking the sortkey for users

5

select "column", type, distkey

6

from pg_table_def where tablename = 'users' and schemaname='public';

7

8

select * from svv_table_info s where s. schema='public' and s.table='users';

9

10

Result 1 (1)

Export

Chart

Row 9, Col 1, Chr 301

database	schema	table_id	table	encoded	diststyle
dev	public	107119	users	Y,AUTO(ENCODE)	KEY(user

Query svv_table_info

```
SELECT table,
       diststyle,
       skew_rows
FROM svv_table_info
WHERE table = 'users';
```

Observation

- Distribution Style: `KEY`
- Distribution Key: `userid`

Check Disk Usage

Query svv_diskusage

The screenshot shows the AWS Redshift Query Editor v2 interface. The left sidebar contains navigation options: Editor, Queries, Notebooks, Charts, History, and Scheduled queries. The main editor area displays a SQL query to check disk usage and distribution for the 'users' table. The query is as follows:

```
1 -- USERID is defined as the DISTKEY column
2
3 -- checking the disk usage, distribution
4 select slice, col, num_values as rows, minvalue, maxvalue
5 from svv_diskusage
6 where name='users' and tbl = (select table_id from svv_table_info s where s.schema='public' and s.table = 'users')
7 and col=0
8 and num_values>0
9 order by slice, col;
```

The query results are displayed in a table with the following data:

slice	col	rows	minvalue	maxvalue
0	0	24958	2	49990
1	0	25032	1	49988

```
SELECT slice,
       num_values,
       minvalue,
       maxvalue
FROM svv_diskusage
WHERE name = 'users';
```

Observation

- Cluster has:
 - 1 node
 - 2 slices

Rows were distributed almost equally across slices.

Meaning

`userid` distributes data efficiently.

Create Table with Different DISTKEY

Create userskey Table

```
CREATE TABLE userskey
DISTSTYLE KEY
DISTKEY(state)
AS SELECT * FROM public.users;
```

Check Disk Usage

The screenshot shows the AWS Redshift Query Editor v2 interface. The left sidebar displays the database structure with a tree view showing 'redshift-cluster-1' > 'native databases (2)' > 'dev' > 'public' > 'Tables' (7). The main editor area contains a SQL query to create a table and check disk usage. The query is as follows:

```
1 -- table is evenly distributed, same range,
2 -- col is userid
3
4 -- state as the dist key
5 create table userskey distkey(state) as select * from users;
6
7 -- checking the disk usage with state as distkey
8 select slice, col, num_values as rows, minvalue, maxvalue
9 from svv_diskusage
10 where name = 'userskey' and tbl = (select table_id from svv_table_info s where s.schema = 'table_design_demo' and s.table = 'userskey')
11 and col = 0
12 and num_values > 0;
```

The query results are displayed in a table with the following data:

slice	col	rows	minvalue	maxvalue
0	0	30368	1	49990
1	0	19622	4	49987

```
SELECT slice,s
       num_values,
       minvalue,
       maxvalue
FROM svv_diskusage
WHERE name = 'userskey';
```

Observation

- Uneven row distribution across slices.

Example:

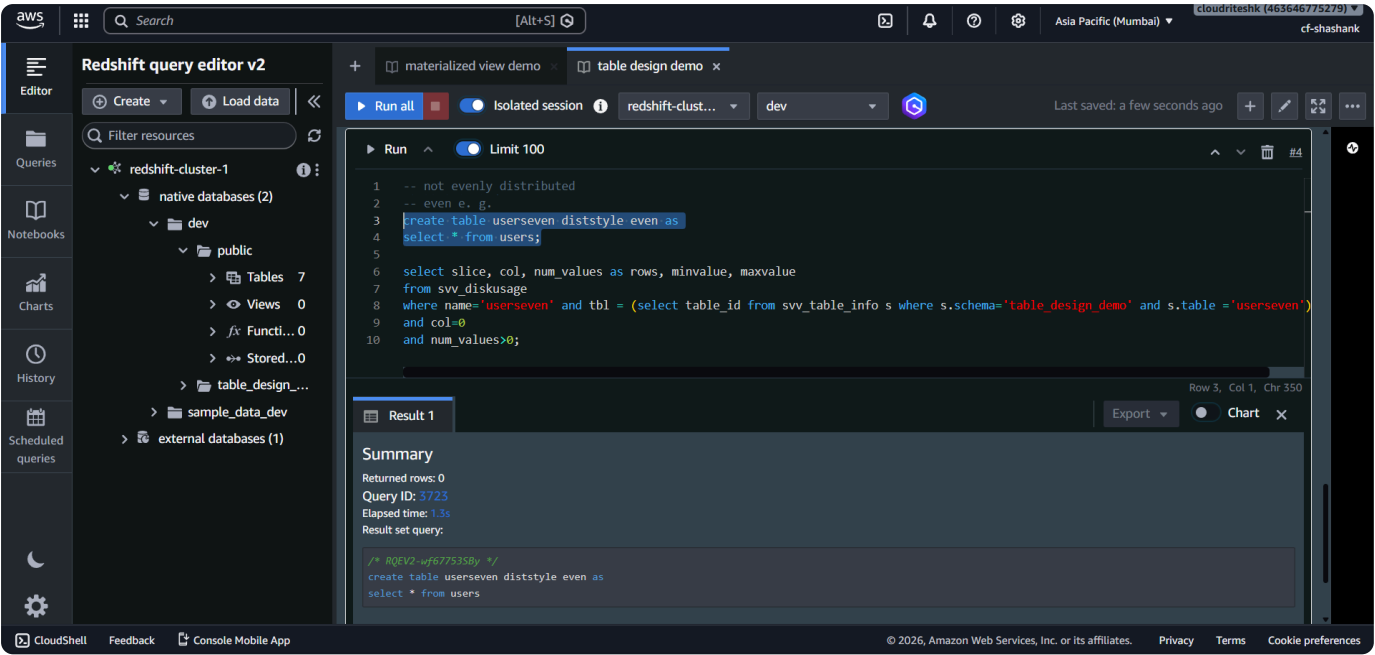
- Slice 0 → ~30000 rows
- Slice 1 → ~19000 rows

Meaning

`state` is not a good distribution key.

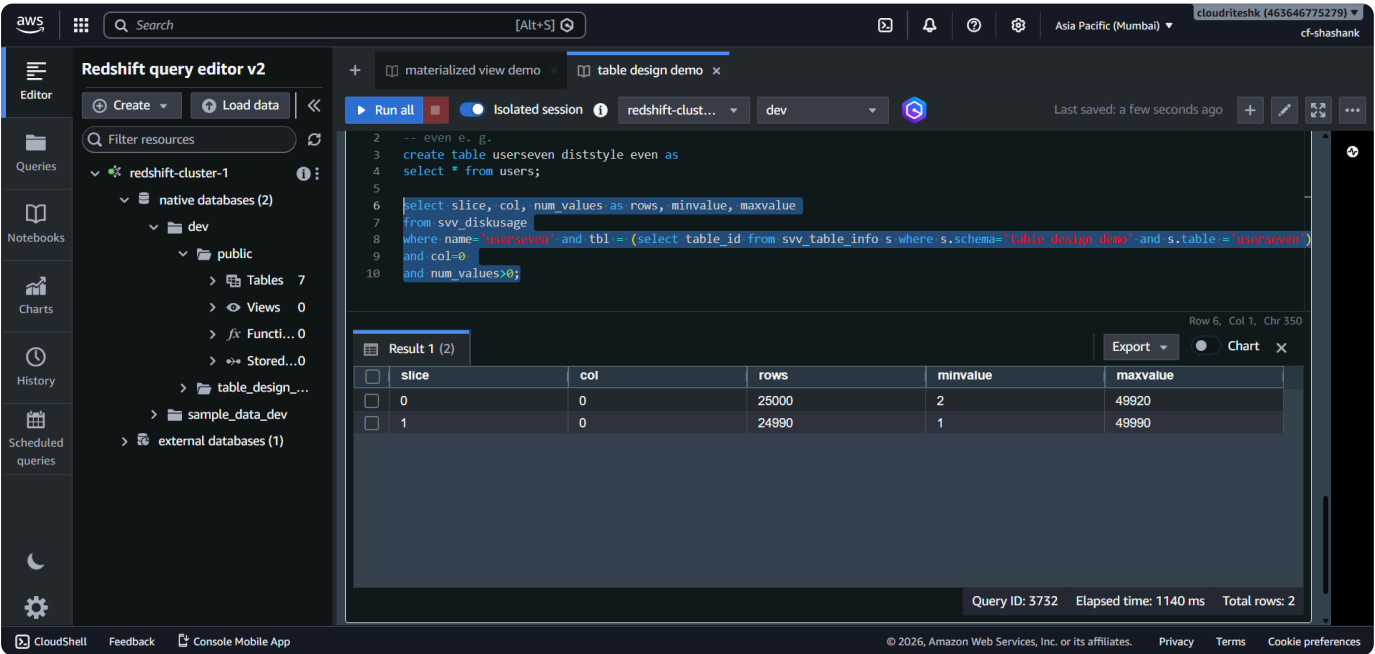
EVEN Distribution Style

Create userseven Table



```
CREATE TABLE userseven
DISTSTYLE EVEN
AS
SELECT * FROM public.users;
```

Check Disk Usage



```
SELECT slice,
       num_values
FROM svv_diskusage
WHERE name = 'userseven';
```

Observation

- Rows distributed almost equally.

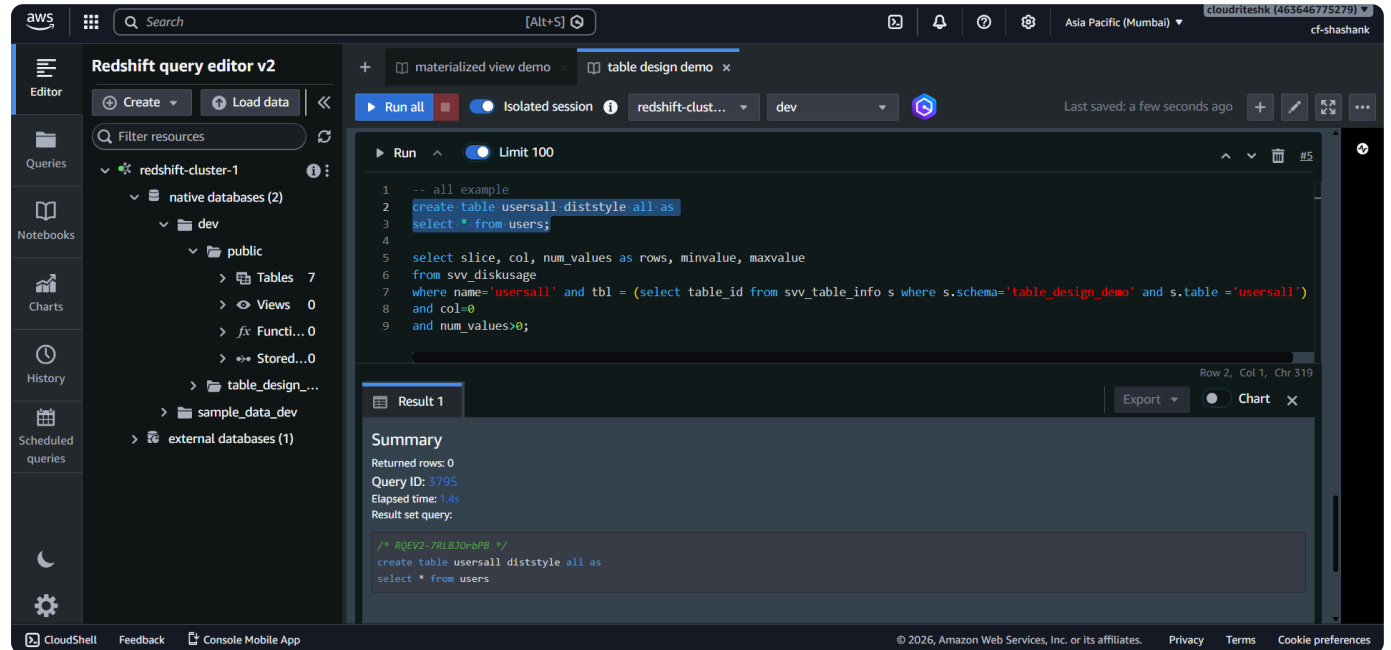
Why?

EVEN distribution uses: `Round Robin`

distribution across slices.

ALL Distribution Style

Create usersall Table



```
CREATE TABLE usersall
DISTSTYLE ALL
AS
SELECT * FROM public.users;
```

Check Disk Usage

The screenshot shows the AWS Redshift Query Editor v2 interface. The left sidebar contains navigation options: Editor, Queries, Notebooks, Charts, History, and Scheduled queries. The main editor area displays a SQL query in a dark theme. The query is as follows:

```
1 -- all example
2 create table usersall diststyle all as
3 select * from users;
4
5 select slice, col, num_values as rows, minvalue, maxvalue
6 from svv_diskusage
7 where name= 'usersall' and tbl = (select table_id from svv_table_info s where s.schema= 'table_design_demo' and s.table = 'usersall')
8 and col=0
9 and num_values>0;
```

Below the query editor, the results are displayed in a table format. The table has 5 columns: slice, col, rows, minvalue, and maxvalue. The results show a single row for slice 0, with 49990 rows and a minvalue of 1 and maxvalue of 49990.

slice	col	rows	minvalue	maxvalue
0	0	49990	1	49990

```
SELECT *
FROM svv_diskusage
WHERE name = 'usersall';
```

Observation

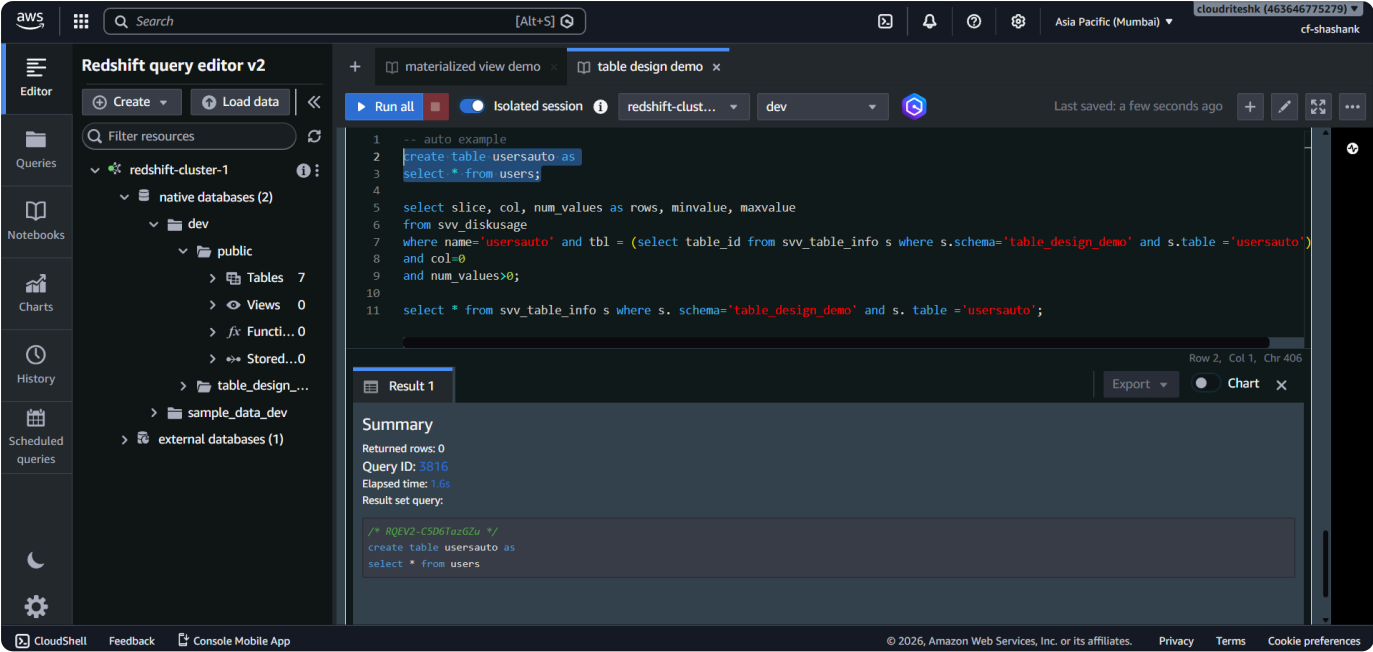
- Full table copied to every node.
- No slice-level distribution.

Best Use Case

- Small tables
- Rarely updated tables

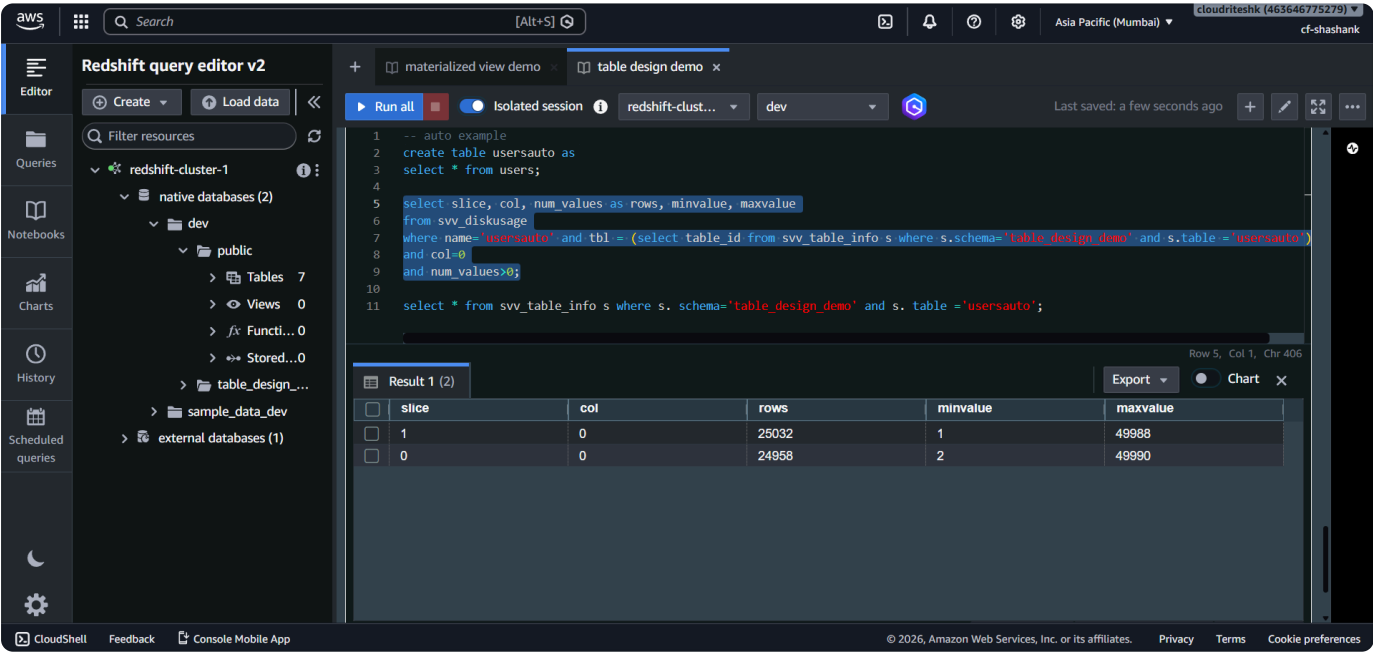
AUTO Distribution Style

Create usersauto Table



```
CREATE TABLE usersauto
AS
SELECT * FROM public.users;
```

Check Distribution Info



```
SELECT table,
       diststyle
FROM svv_table_info
WHERE table = 'usersauto';
```

Observation

Redshift automatically selected:

- Distribution Style: `KEY`
- Distribution Key: `user id`

Distribution Style Summary

Distribution Style	Behavior	Best Use Case
KEY	Distributes using specific column	Joins and balanced queries
EVEN	Round robin distribution	General balanced storage
ALL	Full copy on each node	Small lookup tables
AUTO	Redshift chooses automatically	Recommended default

Key Learning

Distribution style directly impacts:

- Data storage
- Slice balancing
- Query performance

Choosing the wrong distribution key can create uneven storage and slower query execution.